Assignment -10.3

Hall ticket No:2303A51553

Name: M.Dakshinya

Task-01-AI-Assisted Bug Detection

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Prompt:

#Find the logical bug in this Python factorial function, explain it, fix the code, and handle edge cases like 0 and negative numbers.

Code:

#The logical bug in the provided factorial function is that it does not include the number `n` in the multiplication. The loop runs from 1 to `n-1`, which means it will not calculate the factorial correctly for any number greater than 1. Additionally, the function does not handle edge cases such as 0 and negative numbers.

#To fix the code, we need to include `n` in the multiplication and also handle the edge cases. The factorial of 0 is defined as 1, and the factorial of negative numbers is not defined (we can return an error message or raise an exception).

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    elif n == 0:
        return 1
    else:
        result = 1
        for i in range(1, n + 1):
            result *= i
```

```
    return result
# Example usage:
print(factorial(5))
```

Output:

```
(.venv) PS C:\Users\kumar\OneDrive\Desktop\assistant ai> & "C:/Users/kumar/OneDrive/Desktop/assistant ai/.venv/Scripts/pytho
n.exe" "c:/Users/kumar/OneDrive/Desktop/assistant ai/10.3.py"
120
(.venv) PS C:\Users\kumar\OneDrive\Desktop\assistant ai>
```

Explanation:

The given factorial function contains a **logical error in the loop range**. The loop runs from 1 to n-1 instead of including n, causing an **off-by-one error**. Because of this, factorial(5) returns 24 instead of 120.
Using AI, the issue was identified and corrected by changing the loop to range(1, n+1). AI also suggested handling **edge cases such as** 0! = 1 **and negative inputs**, improving the reliability of the function.

Task-02-Improving Readability &Documentation

```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":
    if b == 0:
```

Prompt:

#Review this Python function and rewrite it with better naming, clear docstring, input validation, and division-by-zero exception handling.

Code:

```
def calculate(a, b, operation):
    """
    Perform a basic arithmetic operation on two numbers.
    Parameters:
    a (float): The first number.
    b (float): The second number.
    operation (str): The operation to perform. Can be 'add', 'subtract', 'multiply', or 'divide'.
```

```
    Returns:
    float: The result of the arithmetic operation.
    Raises:
    ValueError: If the operation is not one of the specified options.
    ZeroDivisionError: If division by zero is attempted.
    """
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b == 0:
            raise ZeroDivisionError("Cannot divide by zero.")
        return a / b
    else:
        raise ValueError("Invalid operation. Please choose 'add', 'subtract', 'multiply', or
'divide'.")
# Example usage:
print(calculate(10, 5, 'add'))       # Output: 15
print(calculate(10, 5, 'subtract'))  # Output: 5
print(calculate(10, 5, 'multiply'))  # Output: 50
```

Output:

```
15
5
50
(.venv) PS C:\Users\kumar\OneDrive\Desktop\assistant ai>
⚠ 0                                                    Ln 51
```

Explanation:

The original function had **poor readability**, unclear parameter names (a, b, c), and no documentation. AI suggested improving it by using **descriptive names**, adding a **docstring explaining the function**, and implementing **input validation and exception handling** for division by zero.
These changes make the code **more understandable, maintainable, and safer for real-world usage**.

Task-03-Enforcing Coding Standards

```
def Checkprime(n):

for i in range(2, n):

if n % i == 0:

return False

return True
```

Prompt:

 # Check this Python code for PEP8 violations, list the issues, and provide a refactored PEP8-compliant version.The provided code has several PEP8 violations, including:

Code:

```
#1. The function name should be in lowercase with words separated by underscores (e.g., `check_prime` instead of `Checkprime`).

#2. The code is not properly indented, which can lead to syntax errors.

#3. There should be a blank line after the function definition for better readability.

#Here is the refactored PEP8-compliant version of the code:

def check_prime(n):
    """
    Check if a number is prime.
    Parameters:
    n (int): The number to check.
    Returns:
    bool: True if the number is prime, False otherwise.
```

```
    """
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
# Example usage:
print(check_prime(7))  # Output: True
```

Output:

```
(.venv) PS C:\Users\kumar\OneDrive\Desktop\assistant ai> & "C:/Users/kumar/OneDrive/Desktop/assistant ai/.venv/Scripts/
n.exe" "c:/Users/kumar/OneDrive/Desktop/assistant ai/10.3.py"
120
15
5
50
True
(.venv) PS C:\Users\kumar\OneDrive\Desktop\assistant ai>
```

Explanation:

In **Task 3**, Earlier, the program caused an error when dividing by zero.
The syntax was corrected by adding an if condition to check b == 0 before division.

Task-04-AI as a Code Reviewer in Real Projects

```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

Prompt :

##Act as a senior code reviewer and improve this function with better naming, type hints, input validation, and configurable multiplier.

```
def process_data(data: list, multiplier: int = 2) -> list:
    """
    Process a list of numbers by multiplying even numbers with a specified multiplier.

    Parameters:
    data (list): A list of numbers to process.
```

```
    multiplier (int): The value to multiply even numbers by. Default is 2.

    Returns:
    list: A list of processed numbers where even numbers are multiplied by the multiplier.

    Raises:
    ValueError: If the input data is not a list or if the multiplier is not an integer.
    """

    if not isinstance(data, list):
        raise ValueError("Input data must be a list.")

    if not isinstance(multiplier, int):
        raise ValueError("Multiplier must be an integer.")

    return [x * multiplier for x in data if isinstance(x, (int, float)) and x % 2 == 0]
# Example usage:
print(process_data([1, 2, 3, 4, 5], multiplier=3))  # Output: [6, 12]
```

Output:

```
True
[6, 12]
(.venv) PS C:\Users\kumar\OneDrive\Desktop\assistant ai>
```

Explanation:

The function violated **PEP8 coding standards**, such as improper function naming (Checkprime instead of check_prime) and inconsistent formatting. AI identified these issues and refactored the code to follow **Python naming conventions, spacing, and readability rules**.
Using AI for such checks helps teams **maintain consistent coding standards and reduce manual review effort**

Task-05-AI-Assisted Performance Optimization

```
def sum_of_squares(numbers):
```

```
total = 0

for num in numbers:

total += num ** 2

return total
```

Prompt :

#Analyze the time complexity of this Python function and provide a faster optimized implementation.

Code:

#The time complexity of the provided `sum_of_squares` function is O(n), where n is the number of elements in the input list `numbers`. This is because the function iterates through each element in the list once to calculate the sum of squares.

#A faster optimized implementation can be achieved using a generator expression, which can be more memory efficient and slightly faster due to reduced overhead. Here is the optimized version:

```
def sum_of_squares(numbers):

    return sum(num ** 2 for num in numbers)
```

This implementation still has a time complexity of O(n) but is more concise and can be

#more efficient in terms of memory usage, especially for large lists, as it does not create an intermediate list of squares.

Example usage:

```
print(sum_of_squares([1, 2, 3, 4]))  # Output: 30
```

Output:

Explanation:

The original function calculates the sum of squares using a loop, which works but can be optimized. AI analyzed the ***time complexity (O(n))*** *and suggested using* ***Python's built-in*** *sum()* ***with a generator expression*** *for cleaner and faster execution.*

*This approach improves **performance and readability**, especially when working with large datasets.*